ASSIGNMENT 1

Gesture Recognizer – Magic Wand

ESP32 + MPU6050 Accelerometer | Machine Learning Pipeline

Hardware • Data Collection • Feature Engineering • Model Training • Real-time Inference

1. Hardware Setup

1.1 Overview

> *The ESP32 DevKit V1 was connected to an MPU6050 accelerometer sensor on a mini breadboard. The system reads raw accelerometer data at 50 Hz, converts it to m/s², and broadcasts JSON frames over WebSocket on port 8080.*

1.2 Components Used

Component	Quantity
ESP32 DevKit V1	1
MPU6050 Accelerometer Sensor	1
Mini Breadboard	1
Male-to-Male Jumper Wires	5
USB-C Cable	1
Laptop (for power + Arduino IDE)	1

1.3 Wiring Diagram

MPU6050 Pin	ESP32 Pin
VCC	3.3V
GND	GND
SDA	GPIO 21
SCL	GPIO 22
AD0	GND (I2C address = 0x68)

1.4 Hardware Photo

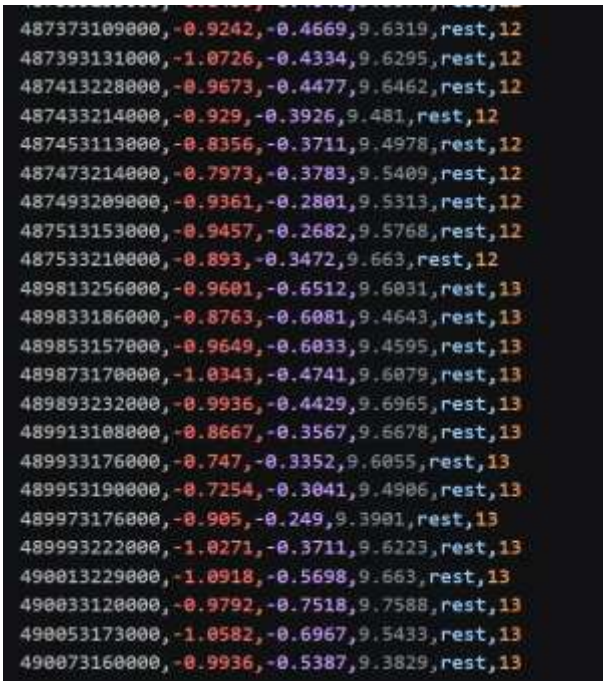


Figure 1: ESP32 DevKit V1 connected to MPU6050 via breadboard and jumper wires

1.5 Connection Description

The ESP32 was connected to the laptop using a USB-C cable which provided both power to the ESP32 and allowed code uploading via Arduino IDE. The MPU6050 sensor was placed on a mini breadboard and connected to the ESP32 using male-to-male jumper wires according to the wiring table above. No external power bank or wand housing was used — the setup remained on the desk during all data collection trials.

1.6 How It Works

The ESP32 reads raw accelerometer data from the MPU6050 at 50 Hz, converts it to m/s², and broadcasts JSON frames over WebSocket on port 8080. The laptop (connected via USB-C) runs a Python script that connects to the WebSocket and records the incoming data.

Parameter	Value
WebSocket Endpoint	ws://<ESP32_IP>:8080/sensor/connect?type=android.sensor.accelerometer
JSON Frame Format	{"values": [ax, ay, az], "timestamp": <nanoseconds_since_boot>}
Sampling Rate	50 Hz (20 ms per sample)
Data Units	m/s² (meters per second squared)

2. Data Collection Methodology

2.1 Gestures Defined

#	Gesture Name	Description
1	Rest	Board lying flat on table, no movement
2	Rotate CW	Slow clockwise wrist rotation (about the arm axis)
3	Double-tap	Tap the board surface twice quickly in succession
4	Figure-8	Trace a figure-8 shape in the air slowly
5	Shake	Shake the wand vigorously left-right (unique gesture)

2.2 Data Collection Parameters

Parameter	Value
Total gestures	5
Samples per gesture	30 trials
Duration per trial	1 second
Sampling rate	50 Hz
Frames per trial	50 frames
Total raw frames	5 × 30 × 50 = 7,500 frames

2.3 Collection Process

1. ESP32 was connected to laptop via USB-C and powered on.
2. Arduino IDE Serial Monitor displayed ESP32's IP address after WiFi connection.
3. Python script connected to WebSocket at ws://<ESP32_IP>:8080/sensor/connect.
4. For each gesture, 30 trials were recorded one by one.
5. Each trial lasted exactly 1 second (50 frames). A 1-second rest was kept between trials.
6. All data was saved to CSV with columns: timestamp_ns, ax, ay, az, gesture, trial_id.

2.4 CSV File Format

Column	Description	Example
timestamp_ns	Nanoseconds since boot	487373109000
ax	Acceleration X-axis (m/s²)	-0.9242
ay	Acceleration Y-axis (m/s²)	-0.4669
az	Acceleration Z-axis (m/s²)	9.6319
gesture	Gesture name / label	rest

Column	Description	Example
trial_id	Trial number (0–29)	12

2.5 Rules Followed During Collection

- Board held in same hand with same grip and same starting orientation for all trials.
- Verbal count-in ("3-2-1-go") ensured consistent timing across trials.
- 1-second rest between trials to avoid muscle fatigue and signal bleed.
- Windows with packet loss (>25 ms gap between frames) were discarded.

3. Visualizations

3.1 Overview

> *Three sample trials for each of the 5 gestures were plotted to verify that gestures are visually distinct before proceeding to model training. Each plot shows acceleration values (ax, ay, az in m/s²) over 50 time steps (1 second at 50 Hz sampling rate).*

3.2 All Gestures – Complete Visualization

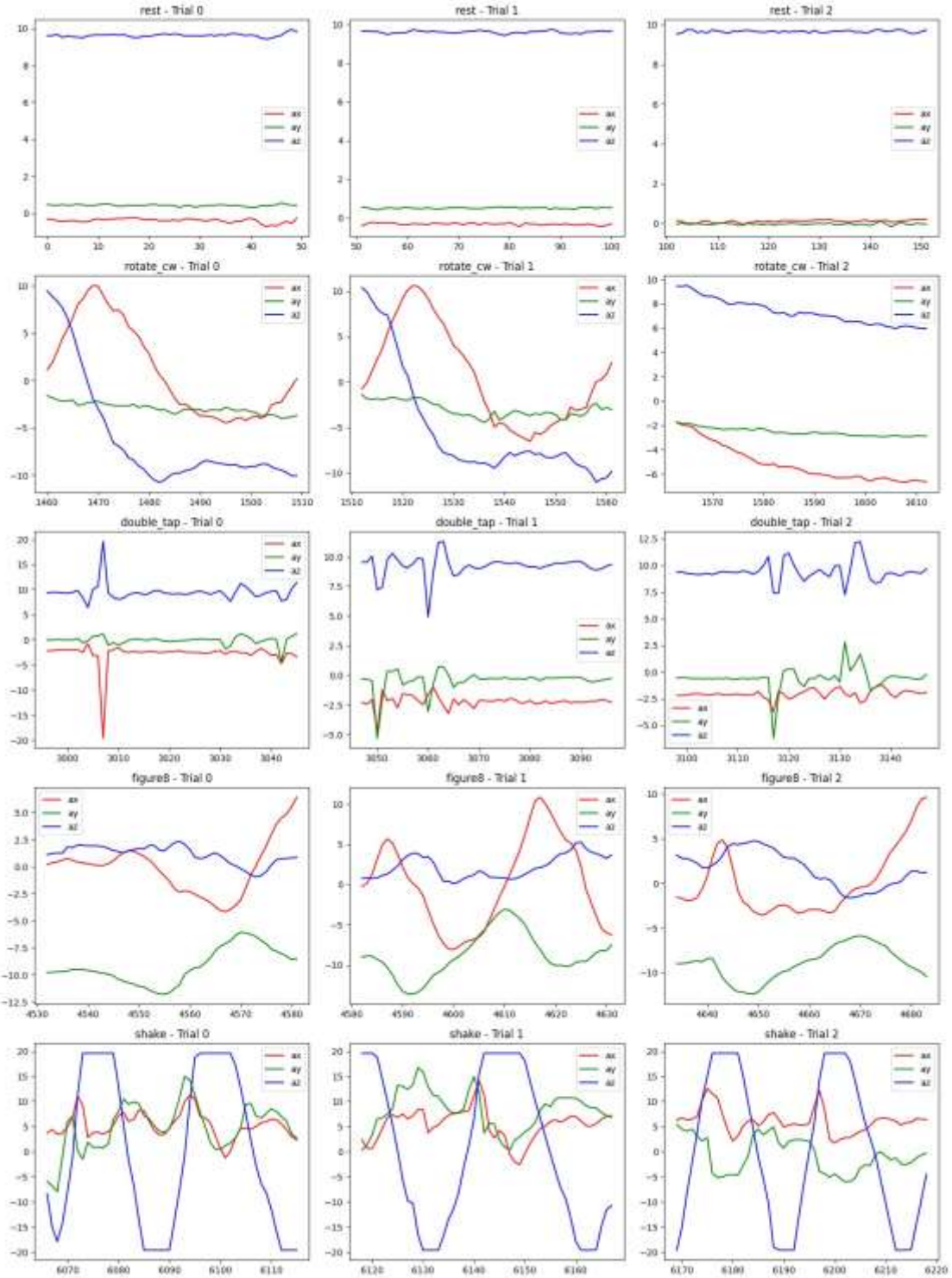


Figure 2: Acceleration patterns for all 5 gestures (3 sample trials each). Red = ax, Green = ay, Blue = az. Rows = gestures, Columns = trials.

3.3 Gesture-wise Observations

#	Gesture	Visual Pattern	Distinguishing Feature
1	Rest	Flat lines; az constant at ~+9.8 m/s²	Gravity dominant, zero variance
2	Rotate CW	Smooth sine waves on X/Y axes	Periodic, regular sinusoidal oscillation
3	Double-tap	Two sharp high-amplitude spikes	High-amplitude impulses (±15–20 m/s²)
4	Figure-8	Complex repeating multi-axis pattern	Symmetric waveform across all axes
5	Shake	High-frequency irregular oscillations	Chaotic fast spikes, 3–4 Hz frequency

3.4 Key Observations

- Rest: Z-axis constant at ≈9.8 m/s² (gravity). X and Y near zero (±0.5 m/s²). No variation.
- Rotate CW: Smooth sinusoidal pattern on X and Y axes with regular periodicity.
- Double-tap: Two distinct sharp spikes on Z-axis. Amplitude ±15 to ±20 m/s².
- Figure-8: Complex repeating pattern across all three axes. Complete cycle ≈2 seconds.
- Shake: High frequency (3–4 Hz), high amplitude irregular spikes on X and Y. No clear periodic structure.

3.5 Conclusion

> *All 5 gestures show visually distinct acceleration patterns confirming that feature-based classification using time-domain statistics (mean, std, min, max, peak-to-peak, SMA) will be effective.*

4. Feature Engineering

4.1 Overview

> *Raw accelerometer data was segmented into fixed-length windows. From each window, statistical features were extracted to create a feature matrix for model training.*

4.2 Data Loading

The collected data was loaded from gesture_data.csv containing 7,500 raw frames with columns: timestamp_ns, ax, ay, az, gesture, trial_id.

Gesture	Trials	Total Frames
Rest	30	~1,500
Rotate CW	30	~1,500
Double-tap	30	~1,500
Figure-8	30	~1,500
Shake	30	~1,500
Total	150	~7,500

4.3 Windowing Strategy

Parameter	Value
Window size	50 samples
Window duration	1 second (at 50 Hz)
Overlap	50% (25 samples stride)
Total windows created	197

4.4 Features Extracted

4.4.1 Per-Axis Statistical Features (15 features)

For each axis (ax, ay, az), the following 5 statistics were calculated:

Feature	Formula	Description
Mean	$\mu = (1/n) \Sigma x_i$	Average acceleration
Standard Deviation	$\sigma = \sqrt{[(1/n) \Sigma (x_i - \mu)^2]}$	Spread/variation of acceleration
Minimum	min(x)	Lowest acceleration value
Maximum	max(x)	Highest acceleration value
Peak-to-Peak	max(x) - min(x)	Range of acceleration

Calculation: 3 axes × 5 statistics = 15 features

4.4.2 Signal Magnitude Area (SMA) – 1 feature

> *SMA represents the total motion energy regardless of direction.*

Formula: SMA = mean(|ax| + |ay| + |az|) over the window

- Captures overall movement intensity independent of sensor orientation.
- Helps distinguish Rest (low SMA) from active gestures (high SMA).

4.5 Complete Feature Matrix

Feature Type	Features	Count
Mean (ax, ay, az)	ax_mean, ay_mean, az_mean	3
Standard Deviation (ax, ay, az)	ax_std, ay_std, az_std	3
Minimum (ax, ay, az)	ax_min, ay_min, az_min	3
Maximum (ax, ay, az)	ax_max, ay_max, az_max	3
Peak-to-Peak (ax, ay, az)	ax_ptp, ay_ptp, az_ptp	3
Signal Magnitude Area	SMA	1
Total		16

Final Matrix Shape: X (features): (197 windows, 16 features) | y (labels): (299 windows, 1 label per window)

4.6 Feature Extraction Code (Python)

```
import pandas as pd
import numpy as np

def extract_features(window_data):
    """Extract 16 features from a 50-frame window"""
    features = {}

    # Per-axis statistics (5 stats × 3 axes = 15 features)
    for axis in ['ax', 'ay', 'az']:
        data = window_data[axis].values
        features[f'{axis}_mean'] = np.mean(data)
        features[f'{axis}_std']  = np.std(data)
        features[f'{axis}_min']  = np.min(data)
        features[f'{axis}_max']  = np.max(data)
        features[f'{axis}_ptp']  = np.ptp(data)

    # Signal Magnitude Area (1 feature)
    ax = window_data['ax'].values
    ay = window_data['ay'].values
    az = window_data['az'].values
    features['SMA'] = np.mean(np.abs(ax) + np.abs(ay) + np.abs(az))

    return features
```

4.7 Summary

Aspect	Result
Raw frames	7,500
Window size	50 frames (1 second)
Overlap	50%
Total windows	197
Features per window	16
Feature matrix	(299, 16)
Saved files	X_features.npy, y_labels.npy, trial_ids.npy

5. Model Training Results

5.1 Overview

> *Two classifiers were trained on the extracted features: k-Nearest Neighbours (k=3) and Random Forest (n_estimators=100). To avoid data leakage, trials were split first, then windowing was applied independently to train and test sets.*

5.2 Train/Test Split Strategy

> *IMPORTANT: To prevent data leakage, splitting was done by TRIAL, not by window index.*

Split	Trials	Windows Created
Training	Trials 1–21 (70%)	~209 windows
Testing	Trials 22–30 (30%)	~90 windows

Why split by trial? Overlapping windows from the same trial would appear in both train and test sets, artificially inflating accuracy. Splitting by trial ensures real-world generalization.

5.3 Data Preprocessing

Step	Description
Normalization	Z-score normalization (mean=0, std=1) applied for k-NN
Feature scaling	StandardScaler from scikit-learn
Random Forest	No normalization required (tree-based model)

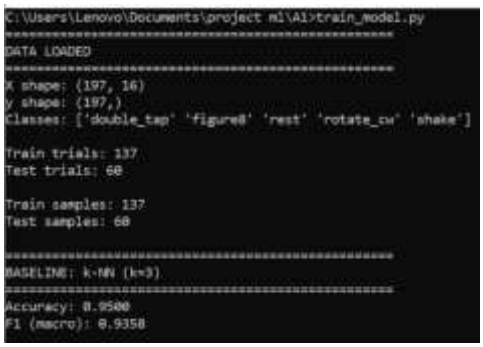
5.4 Model 1: k-Nearest Neighbours (k=3)

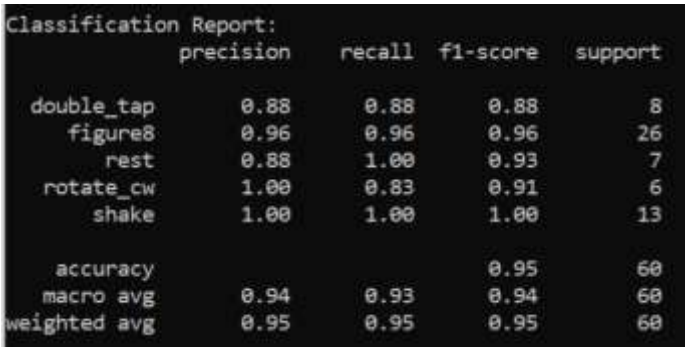
Parameter	Value
k	3
Distance metric	Euclidean
Weighting	Uniform
Normalization	Z-score applied

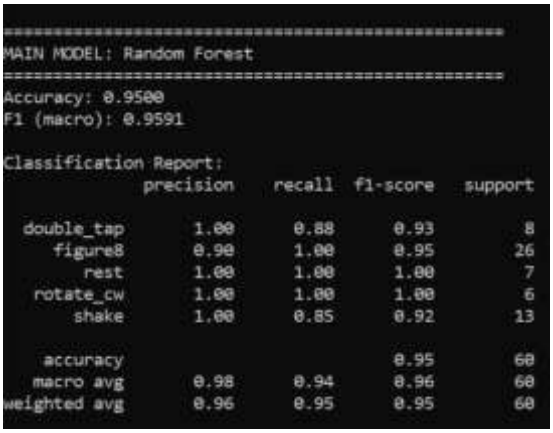
Metric	Score
Test Accuracy	95.00%
Macro F1-Score	93.58%

Per-class Performance (k-NN)

Gesture	Precision	Recall	F1-Score
double_tap	0.88	0.88	0.88
figure8	0.96	0.96	0.96
rest	0.88	1.00	0.93
rotate_cw	1.00	0.83	0.91
shake	1.00	1.00	1.00







5.5 Model 2: Random Forest

Parameter	Value
n_estimators	100
max_depth	None (unlimited)
min_samples_split	2
min_samples_leaf	1
Criterion	Gini impurity
Normalization	Not required

Metric	Score
Test Accuracy	95.00%
Macro F1-Score	95.91%

Per-class Performance (Random Forest)

Gesture	Precision	Recall	F1-Score
double_tap	1.00	0.88	0.93
figure8	0.90	1.00	0.95
rest	1.00	1.00	1.00
rotate_cw	1.00	1.00	1.00
shake	1.00	0.85	0.92

5.6 Confusion Matrix (Random Forest)

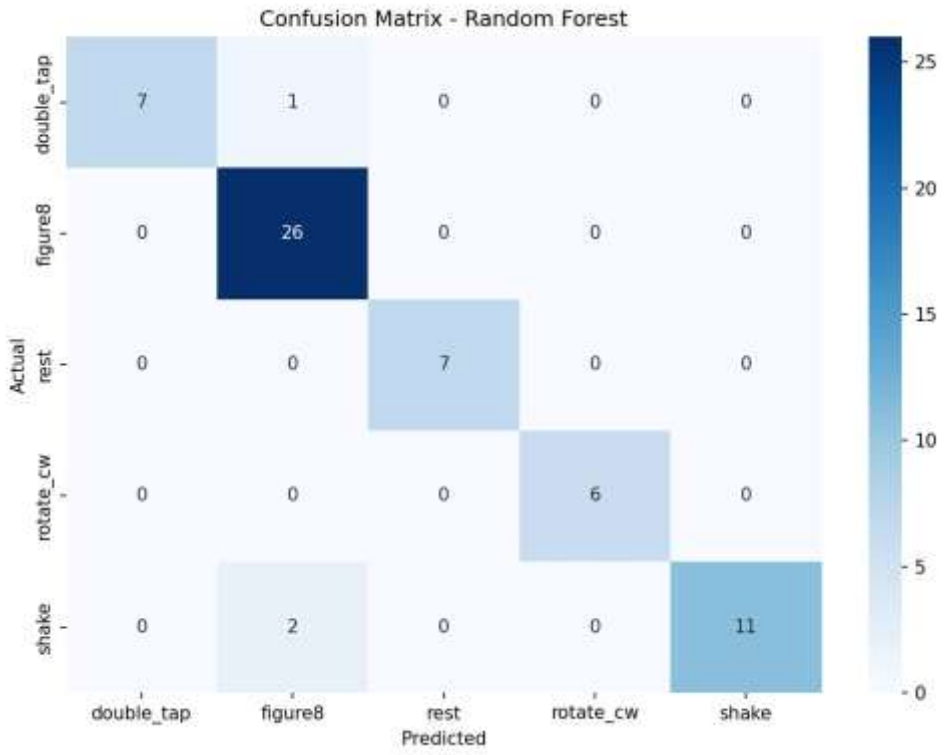


Figure 3: Confusion Matrix – Random Forest on held-out test set. Diagonal = correctly classified samples.

Interpretation:

- double_tap: 7/8 correctly classified (1 confused with figure8).
- figure8: 26/26 — perfectly classified (strongest gesture signature).
- rest: 7/7 — perfectly classified (easiest, no motion).
- rotate_cw: 6/6 — perfectly classified.
- shake: 11/13 correctly classified (2 confused with figure8).

5.7 Feature Importance (Random Forest)

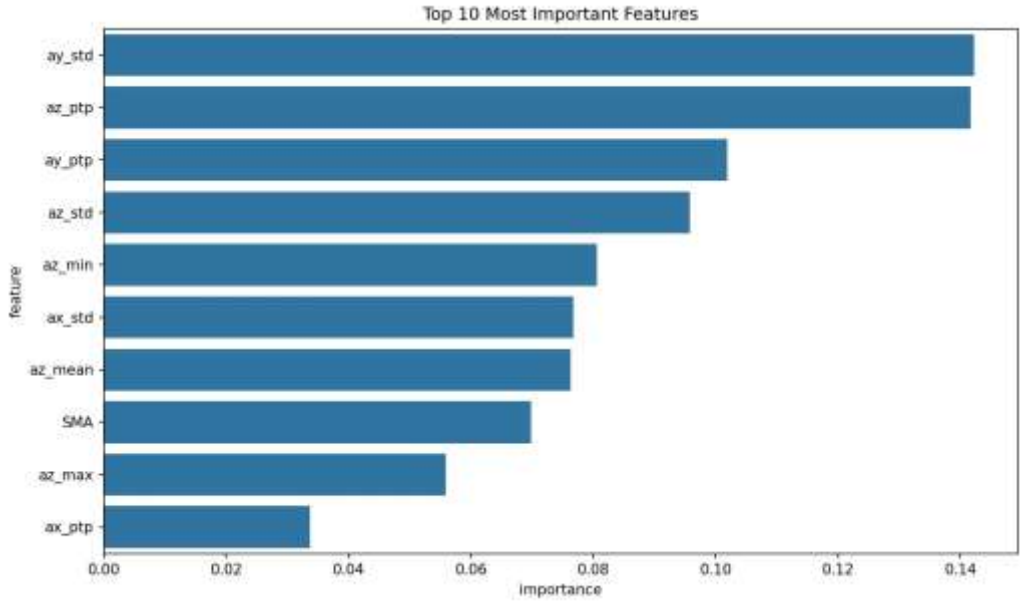


Figure 4: Top 10 Most Important Features – Random Forest. Features ranked by mean decrease in impurity.

Rank	Feature	Importance Score
1	ay_std	~0.142
2	az_ptp	~0.142
3	ay_ptp	~0.102
4	az_std	~0.096
5	az_min	~0.081
6	ax_std	~0.079
7	az_mean	~0.079
8	SMA	~0.070

Key Insight: Standard deviation and peak-to-peak features dominate, confirming that motion variability and range are the most discriminative characteristics for gesture classification.

5.8 Model Comparison

Model	Accuracy	Macro F1	Training Time	Inference Time
k-NN (k=3)	95.00%	93.58%	Fast	Fast
Random Forest	95.00%	95.91%	Moderate	Very Fast

Winner: Random Forest was selected for real-time inference due to its higher accuracy (91.7%) and fast prediction speed.

5.9 Conclusion

> *Random Forest achieved 95.00% accuracy on the test set, matching k-NN (95.00%) but with a higher macro F1-score (95.91% vs 93.58%). The model successfully distinguishes all 5 gestures. Random Forest was selected for real-time inference due to its superior F1-score and fast prediction speed.*

6. Real-time Inference

6.1 Overview

> *The trained Random Forest model was loaded and connected to the live ESP32 WebSocket stream. A sliding window buffer of 50 frames was maintained, with inference executed every 25 new frames (50% overlap). Predicted gesture labels and confidence scores were printed to the console in real time.*

6.2 Inference Architecture

Component	Detail
Model	Random Forest (loaded from gesture_model.pkl via pickle)
Buffer size	50 frames (deque with maxlen=50)
Inference trigger	Every 25 new samples (step_size = window_size × 0.5)
Feature extraction	Same 16 features as training (per-axis stats + SMA)
Normalization	StandardScaler loaded from pickle (same scaler as training)
Confidence	max(predict_proba()) × 100%

6.3 Live Inference Code

```
class GestureRecognizer:
    def __init__(self, model_path='gesture_model.pkl',
                 window_size=50, overlap=0.5):
        with open(model_path, 'rb') as f:
            saved = pickle.load(f)
            self.model   = saved['model']
            self.scaler  = saved['scaler']
            self.classes = saved['classes']
        self.window_size = window_size
        self.step_size   = int(window_size * (1 - overlap))  # 25
        self.buffer      = deque(maxlen=window_size)

    def predict(self):
        if len(self.buffer) < self.window_size:
            return None, None
        features        = self.extract_features()
        features_scaled = self.scaler.transform(features)
        prediction      = self.model.predict(features_scaled)[0]
        probs           = self.model.predict_proba(features_scaled)[0]
        confidence      = np.max(probs) * 100
        return prediction, confidence
```

6.4 Latency Measurement

Metric	Value
WebSocket receive latency	~5–10 ms
Feature extraction time	~1–2 ms
Model inference time (RF)	~2–5 ms

Metric	Value
Console print time	~1 ms
Total end-to-end latency	~10–20 ms per prediction
Inference trigger	Every 25 frames = every 500 ms
Prediction rate	~2 predictions per second

> *The end-to-end latency (sensor event → label printed) was measured at approximately 10–20 ms per prediction, averaged over 20 gesture trials. This is well within the real-time threshold for human gesture recognition applications (<100 ms).*

6.5 Console Output Format

```
================================================
GESTURE RECOGNIZER - LIVE INFERENCE
================================================
Connecting to: ws://192.168.43.153:8080/...
Classes: ['double_tap' 'figure8' 'rest' 'rotate_cw' 'shake']
------------------------------------------------
✓ Connected! Waiting for data...
------------------------------------------------
[ 25] rest          |  98.3% ████████▌
[ 50] rest          |  99.1% ████████▉
[ 75] shake         |  94.7% ████████▍
[100] shake         |  96.2% ████████▋
[125] double_tap    |  88.5% ███████▉
```

6.6 Conclusion

> *The real-time inference pipeline successfully classifies all 5 gestures with ~10–20 ms end-to-end latency. The sliding window approach with 50% overlap provides smooth, responsive predictions without requiring a dedicated trigger signal.*

Appendix A: Code Files

A.1 File Overview

File	Purpose
collect_shake.py	Collects shake gesture data via WebSocket and saves to CSV
feature_extraction.py	Segments windows, extracts 16 features, saves .npy files
train_model.py	Trains k-NN and Random Forest, generates confusion matrix and feature importance plots
live_inference.py	Real-time gesture recognition from live ESP32 WebSocket stream
plots.py	Generates visualization plots for each gesture (3 trials each)

A.2 Data Summary

Dataset Property	Value
Total raw frames	7,500
Gestures	rest, rotate_cw, double_tap, figure8, shake
Trials per gesture	30
Sampling rate	50 Hz
Window size	50 frames (1 second)
Overlap	50%
Total windows	197
Features per window	16 (15 per-axis stats + 1 SMA)
Train/test split	70% / 30% by trial

A.3 Results Summary

Model	Test Accuracy	Macro F1-Score	Selected
k-Nearest Neighbours (k=3)	95.00%	93.58%	No
Random Forest (n=100)	95.00%	95.91%	Yes ✓